

TASK 2

```
In [ ]: import pandas as pd
import numpy as np
```

```
In [8]: quantum = pd.read_csv("/Users/thiernodicko/Desktop/Panda_files/QVI_finaldata.csv")
```

```
In [9]: quantum
```

```
Out[9]:
```

	Unnamed: 0	LYLTY_CARD_NBR	DATE	STORE_NBR	TXN_ID	PROD_NBR	PROD_NAME
0	0	47142.0	2018-07-01	47.0	42540.0	14.0	Smiths Cr Chip Orgn
1	1	28024.0	2018-07-01	28.0	24672.0	113.0	Twisties Chic
2	2	33107.0	2018-07-01	33.0	30115.0	105.0	Woolwo Cheese R
3	3	40130.0	2018-07-01	40.0	36721.0	63.0	Kettle Swt Pot
4	4	45109.0	2018-07-01	45.0	41028.0	79.0	Smiths Cr TF CutSaltVir
...	...	...	...	...	...	...	...
246735	246735	107109.0	2019-06-30	107.0	108968.0	113.0	Twisties Chic
246736	246736	119271.0	2019-06-30	119.0	123199.0	9.0	Kettle To ChpsBtrootRic
246737	246737	122230.0	2019-06-30	122.0	125994.0	106.0	Natural Chi Hony Soy Ch
246738	246738	213140.0	2019-06-30	213.0	212816.0	90.0	Tostitos Smc Chip
246739	246739	265006.0	2019-06-30	265.0	263307.0	106.0	Natural Chi Hony Soy Ch

246740 rows × 13 columns

```
In [10]: quantum.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 246740 entries, 0 to 246739
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            246740 non-null  int64
1   LYLTY_CARD_NBR       246740 non-null  float64
2   DATE                 246740 non-null  object
3   STORE_NBR           246740 non-null  float64
4   TXN_ID              246740 non-null  float64
5   PROD_NBR            246740 non-null  float64
6   PROD_NAME           246740 non-null  object
7   PROD_QTY            246740 non-null  float64
8   TOT_SALES           246740 non-null  float64
9   PACK_SIZE_g         246740 non-null  int64
10  BRAND_NAME          246740 non-null  object
11  LIFESTAGE           246740 non-null  object
12  MEMBER_TYPE         246740 non-null  object
dtypes: float64(6), int64(2), object(5)
memory usage: 24.5+ MB
```

```
In [13]: # Changing the datetime format from object to date
quantium['DATE'] = pd.to_datetime(quantium['DATE'])
```

```
In [14]: # Verify update
quantium.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 246740 entries, 0 to 246739
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            246740 non-null  int64
1   LYLTY_CARD_NBR       246740 non-null  float64
2   DATE                 246740 non-null  datetime64[ns]
3   STORE_NBR           246740 non-null  float64
4   TXN_ID              246740 non-null  float64
5   PROD_NBR            246740 non-null  float64
6   PROD_NAME           246740 non-null  object
7   PROD_QTY            246740 non-null  float64
8   TOT_SALES           246740 non-null  float64
9   PACK_SIZE_g         246740 non-null  int64
10  BRAND_NAME          246740 non-null  object
11  LIFESTAGE           246740 non-null  object
12  MEMBER_TYPE         246740 non-null  object
dtypes: datetime64[ns](1), float64(6), int64(2), object(4)
memory usage: 24.5+ MB
```

```
In [15]: # Add a new month ID column in the data with the format yyyy-mm
quantium['YEARMONTH'] = quantium['DATE'].dt.strftime('%Y%m').astype('int64')
```

```
In [16]: # Display first 5 rows
quantium.head(5)
```

Out[16]:

	Unnamed: 0	LYLTY_CARD_NBR	DATE	STORE_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD.
0	0	47142.0	2018-07-01	47.0	42540.0	14.0	Smiths Crnkle Chip Orgnl Big Bag	
1	1	28024.0	2018-07-01	28.0	24672.0	113.0	Twisties Chicken	
2	2	33107.0	2018-07-01	33.0	30115.0	105.0	Woolworths Cheese Rings	
3	3	40130.0	2018-07-01	40.0	36721.0	63.0	Kettle Swt Pot Sea Salt	
4	4	45109.0	2018-07-01	45.0	41028.0	79.0	Smiths Chip Thinly CutSaltVinegr	

Now for each store and month calculate total sales, number of customers, transactions per customer, chips per customer and the average price per unit.

```
In [17]: # Define metric
groupby_quantum = quantum.groupby(['STORE_NBR', 'YEARMONTH'])
```

```
In [18]: # Total Sales
totSales = groupby_quantum.TOT_SALES.sum()
# Number of customers
nCustomers = groupby_quantum.LYLT_CARD_NBR.nunique()
# Number of Transaction Per Customer
nTxnPerCust = groupby_quantum.TXN_ID.size()/nCustomers
# Number of Chips Per Transaction
nChipsPerTxn = groupby_quantum.PROD_QTY.sum() / groupby_quantum.TXN_ID.size()
# Average Price Per Unit
avgPricePerUnit = totSales/groupby_quantum.PROD_QTY.sum()
```

Arranging the created metrics into an array

```
In [19]: metrics = [totSales, nCustomers, nTxnPerCust, nChipsPerTxn]
```

```
In [20]: # Create a dataframe for the metrics array
metrics = pd.concat(metrics, axis = 1)
```

```
In [21]: # Add the column name respectively as the name in the metrics array
metrics.columns = ["totSales", "nCustomers", "nTxnPerCust", "nChipsPerTxn"]
```

```
In [22]: metrics = metrics.reset_index()
```

```
In [23]: metrics
```

Out [23]:

	STORE_NBR	YEARMONTH	totSales	nCustomers	nTxnPerCust	nChipsPerTxn
0	1.0	201807	188.9	47	1.042553	1.183673
1	1.0	201808	168.4	41	1.000000	1.268293
2	1.0	201809	268.1	57	1.035088	1.203390
3	1.0	201810	175.4	39	1.025641	1.275000
4	1.0	201811	184.8	44	1.022727	1.222222
...	...	...	...	...	...	...
3160	272.0	201902	385.3	44	1.068182	1.893617
3161	272.0	201903	421.9	48	1.062500	1.901961
3162	272.0	201904	445.1	54	1.037037	1.875000
3163	272.0	201905	314.6	34	1.176471	1.775000
3164	272.0	201906	301.9	33	1.090909	1.888889

3165 rows × 6 columns

Filter to the pre-trial period and stores with full observation periods

In [24]:

```
# Filter to select stores with full observation period
month_counts = metrics.groupby('STORE_NBR').YEARMONTH.nunique().reset_index()
stores_fullobs = month_counts[month_counts.YEARMONTH == 12].STORE_NBR
pretrial_metrics = metrics[metrics['STORE_NBR'].isin(stores_fullobs)]

# Then filter to keep only the pre-trial period data
pretrial_metrics = pretrial_metrics.loc[pretrial_metrics.YEARMONTH < 201902]
pretrial_metrics
```

Out [24]:

	STORE_NBR	YEARMONTH	totSales	nCustomers	nTxnPerCust	nChipsPerTxn
0	1.0	201807	188.9	47	1.042553	1.183673
1	1.0	201808	168.4	41	1.000000	1.268293
2	1.0	201809	268.1	57	1.035088	1.203390
3	1.0	201810	175.4	39	1.025641	1.275000
4	1.0	201811	184.8	44	1.022727	1.222222
...	...	...	...	...	...	...
3155	272.0	201809	294.5	31	1.129032	1.971429
3156	272.0	201810	405.1	41	1.170732	1.958333
3157	272.0	201811	355.8	39	1.102564	1.930233
3158	272.0	201812	363.1	43	1.000000	1.883721
3159	272.0	201901	392.4	44	1.068182	1.914894

1813 rows × 6 columns

Let calculate how correlated the performance of each store is to the trial store.

```
In [25]: # Creating a function to calculate correlation for a measure, looping through
def calc_corr(trial, metric_col, input_table = pretrial_metrics):
    trial_stores = [77, 86, 88]
    control_stores = stores_fullobj[~stores_fullobj.isin(trial_stores)] # all
    # Keep the trial store values to perform correlation with
    trial_vals = input_table[input_table["STORE_NBR"] == trial][metric_col].res
    corr_table = pd.DataFrame(columns = ['YEARMONTH', 'trial_store', 'control_
    # Find the correlation for each control store
    for control in control_stores:
        # Keep the control store values to perform correlation with
        control_vals = input_table[input_table["STORE_NBR"] == control][metric
        corr_row = pd.DataFrame(columns = ['YEARMONTH', 'trial_store', 'contro
        corr_row.YEARMONTH = list(input_table.loc[input_table.STORE_NBR == con
        corr_row.trial_store = trial
        corr_row.control_store = control
        corr_row.correlation = control_vals.corrwith(trial_vals, axis=1)
        corr_table = pd.concat([corr_table, corr_row]) # add each store's block
    return (corr_table)
```

```
In [26]: trial_stores = [77, 86, 88]
corr_table = pd.DataFrame(columns = ['YEARMONTH', 'trial_store', 'control_store
for store in trial_stores:
    corr_section = calc_corr(store, ['totSales', 'nCustomers', 'nTxnPerCust',
    corr_table = pd.concat([corr_table, corr_section])
```

```
/var/folders/ly/5zv7mxn60s_ykmn7h875w4w0000gn/T/ipykernel_27452/518383554.py:
17: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA
A entries is deprecated. In a future version, this will no longer exclude empty
y or all-NA columns when determining the result dtypes. To retain the old beha
vior, exclude the relevant entries before the concat operation.
    corr_table = pd.concat([corr_table, corr_row]) # add each store's block to t
he dataframe
/var/folders/ly/5zv7mxn60s_ykmn7h875w4w0000gn/T/ipykernel_27452/2859872201.p
y:5: FutureWarning: The behavior of DataFrame concatenation with empty or all-
NA entries is deprecated. In a future version, this will no longer exclude empty
y or all-NA columns when determining the result dtypes. To retain the old beha
vior, exclude the relevant entries before the concat operation.
    corr_table = pd.concat([corr_table, corr_section])
/var/folders/ly/5zv7mxn60s_ykmn7h875w4w0000gn/T/ipykernel_27452/518383554.py:
17: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA
A entries is deprecated. In a future version, this will no longer exclude empty
y or all-NA columns when determining the result dtypes. To retain the old beha
vior, exclude the relevant entries before the concat operation.
    corr_table = pd.concat([corr_table, corr_row]) # add each store's block to t
he dataframe
/var/folders/ly/5zv7mxn60s_ykmn7h875w4w0000gn/T/ipykernel_27452/518383554.py:
17: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA
A entries is deprecated. In a future version, this will no longer exclude empty
y or all-NA columns when determining the result dtypes. To retain the old beha
vior, exclude the relevant entries before the concat operation.
    corr_table = pd.concat([corr_table, corr_row]) # add each store's block to t
he dataframe
```

```
In [27]: # Displaying the correlation table
corr_table
```

Out [27]:

	YEARMONTH	trial_store	control_store	correlation
0	201807	77	1.0	-0.033919
1	201808	77	1.0	-0.051668
2	201809	77	1.0	-0.074818
3	201810	77	1.0	-0.095846
4	201811	77	1.0	-0.060864
...	...	...	...	...
2	201809	88	272.0	0.519915
3	201810	88	272.0	0.585601
4	201811	88	272.0	0.556683
5	201812	88	272.0	0.599680
6	201901	88	272.0	0.606869

5376 rows x 4 columns

Standardise the magnitude distance so that the measure ranges from 0 to 1

```
In [29]: def calc_magdist(trial, metric_col, input_table = pretrial_metrics):
    trial_stores = [77, 86, 88]
    control_stores = stores_fullobs[~stores_fullobs.isin(trial_stores)] # all
    dist_table = pd.DataFrame() # to store the distances for each store and month
    for control in control_stores: # calculate for each control store
        dist_row = pd.DataFrame()
        # Calculate the distance as an absolute value
        dist_row = abs(input_table[input_table["STORE_NBR"] == trial].reset_index()
                        - input_table[input_table["STORE_NBR"] == control].reset_index())
        dist_row.insert(0, 'YEARMONTH', list(input_table.loc[input_table.STORE_NBR == trial].YEARMONTH))
        dist_row.insert(1, 'trial_store', trial)
        dist_row.insert(2, 'control_store', control)
        dist_table = pd.concat([dist_table, dist_row])

    for col in metric_col: # then loop over each column to find the max and min
        maxdist = dist_table[col].max()
        mindist = dist_table[col].min()
        dist_table[col] = 1-(dist_table[col] - mindist)/(maxdist-mindist) # normalize
        # also give an average magnitude over all metrics per month and store
    dist_table['mag_measure'] = dist_table[metric_col].mean(axis=1)
    return (dist_table)
```

Now let's use the functions to find the control stores! We'll select control stores based on how similar monthly total sales in dollar amounts and monthly number of customers are to the trial stores. So we will need to use our functions to get four scores, two for each of total sales and total customers.

```
In [31]: def calc_corrdist_score (trial, metric_col, input_table=pretrial_metrics):
    # Calculate the correlations and magnitudes for all months
    corr_vals = calc_corr(trial, metric_col, input_table)
    mag_vals = calc_magdist(trial, metric_col, input_table)
    mag_vals = mag_vals.drop(metric_col, axis=1) # For one metric, the two columns
```

```

# Combine correlations and magnitudes together to one df
combined_corr_dist = pd.merge(corr_vals, mag_vals, on=["YEARMONTH", "trial_store"])

# Average correlations and distances over the pre-trial months
avg_corrmag = combined_corr_dist.groupby(["trial_store", "control_store"])

# Find a combined score by taking the weighted average of the correlations
corr_weight = 0.5
avg_corrmag['combined_score'] = corr_weight*avg_corrmag['correlation'] + (1-corr_weight)*avg_corrmag['mag_measure']

return(avg_corrmag)

```

Let find the magnitude with the highest averaged scores.

```

In [32]: def find_highestscore(trial):
# Obtain the scores for the tot_sales and n_cust metrics separately
scores_tot_sales = calc_corrdist_score(trial, ['totSales'])
scores_n_cust = calc_corrdist_score(trial, ['nCustomers'])
# Create a data table to store the composite results in - stores are also
scores_control = pd.DataFrame()
scores_control['control_store'] = scores_tot_sales.control_store
# Calculate the composite scores
scores_control['correlation'] = 0.5*scores_tot_sales.correlation + 0.5*scores_n_cust.correlation
scores_control['mag_measure'] = 0.5*scores_tot_sales.mag_measure + 0.5*scores_n_cust.mag_measure
scores_control['scores'] = 0.5*scores_tot_sales.combined_score + 0.5*scores_n_cust.combined_score
return(scores_control.sort_values(by = 'scores', ascending = False).reset_index())

```

```

In [33]: # Now find the control stores with the highest scores for each of the trial stores
trial_stores = [77, 86, 88]
result = []
for trial in trial_stores:
    print('Trial store: ', trial)
    print(find_highestscore(trial))
    result.append([trial, find_highestscore(trial)])
    print()

```

Trial store: 77

```

/var/folders/ly/5zvv7mxn60s_ykmm7h875w4w0000gn/T/ipykernel_27452/518383554.py:
17: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA
A entries is deprecated. In a future version, this will no longer exclude empty
y or all-NA columns when determining the result dtypes. To retain the old behavior,
exclude the relevant entries before the concat operation.

```

```

corr_table = pd.concat([corr_table, corr_row]) # add each store's block to the dataframe

```

```

/var/folders/ly/5zvv7mxn60s_ykmm7h875w4w0000gn/T/ipykernel_27452/518383554.py:
17: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA
A entries is deprecated. In a future version, this will no longer exclude empty
y or all-NA columns when determining the result dtypes. To retain the old behavior,
exclude the relevant entries before the concat operation.

```

```

corr_table = pd.concat([corr_table, corr_row]) # add each store's block to the dataframe

```

	control_store	correlation	mag_measure	scores
0	233.0	1.0	0.989108	0.994554
1	46.0	1.0	0.967704	0.983852
2	188.0	1.0	0.963409	0.981705
3	53.0	1.0	0.962892	0.981446
4	264.0	1.0	0.961240	0.980620

```
/var/folders/ly/5zv7mxn60s_ykmn7h875w4w0000gn/T/ipykernel_27452/518383554.py:
17: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA
entries is deprecated. In a future version, this will no longer exclude empty
or all-NA columns when determining the result dtypes. To retain the old behavior,
exclude the relevant entries before the concat operation.
```

```
corr_table = pd.concat([corr_table, corr_row]) # add each store's block to the
dataframe
```

```
/var/folders/ly/5zv7mxn60s_ykmn7h875w4w0000gn/T/ipykernel_27452/518383554.py:
17: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA
entries is deprecated. In a future version, this will no longer exclude empty
or all-NA columns when determining the result dtypes. To retain the old behavior,
exclude the relevant entries before the concat operation.
```

```
corr_table = pd.concat([corr_table, corr_row]) # add each store's block to the
dataframe
```

Trial store: 86

```
/var/folders/ly/5zv7mxn60s_ykmn7h875w4w0000gn/T/ipykernel_27452/518383554.py:
17: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA
entries is deprecated. In a future version, this will no longer exclude empty
or all-NA columns when determining the result dtypes. To retain the old behavior,
exclude the relevant entries before the concat operation.
```

```
corr_table = pd.concat([corr_table, corr_row]) # add each store's block to the
dataframe
```

```
/var/folders/ly/5zv7mxn60s_ykmn7h875w4w0000gn/T/ipykernel_27452/518383554.py:
17: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA
entries is deprecated. In a future version, this will no longer exclude empty
or all-NA columns when determining the result dtypes. To retain the old behavior,
exclude the relevant entries before the concat operation.
```

```
corr_table = pd.concat([corr_table, corr_row]) # add each store's block to the
dataframe
```

	control_store	correlation	mag_measure	scores
0	155.0	1.0	0.969600	0.984800
1	109.0	1.0	0.953236	0.976618
2	225.0	1.0	0.950692	0.975346
3	114.0	1.0	0.950586	0.975293
4	184.0	1.0	0.949708	0.974854

```
/var/folders/ly/5zv7mxn60s_ykmn7h875w4w0000gn/T/ipykernel_27452/518383554.py:
17: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA
entries is deprecated. In a future version, this will no longer exclude empty
or all-NA columns when determining the result dtypes. To retain the old behavior,
exclude the relevant entries before the concat operation.
```

```
corr_table = pd.concat([corr_table, corr_row]) # add each store's block to the
dataframe
```

```
/var/folders/ly/5zv7mxn60s_ykmn7h875w4w0000gn/T/ipykernel_27452/518383554.py:
17: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA
entries is deprecated. In a future version, this will no longer exclude empty
or all-NA columns when determining the result dtypes. To retain the old behavior,
exclude the relevant entries before the concat operation.
```

```
corr_table = pd.concat([corr_table, corr_row]) # add each store's block to the
dataframe
```

Trial store: 88

```

/var/folders/ly/5zvv7mxn60s_ykmn7h875w4w0000gn/T/ipykernel_27452/518383554.py:
17: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA
entries is deprecated. In a future version, this will no longer exclude empty
or all-NA columns when determining the result dtypes. To retain the old behavior,
exclude the relevant entries before the concat operation.
    corr_table = pd.concat([corr_table, corr_row]) # add each store's block to the
dataframe
/var/folders/ly/5zvv7mxn60s_ykmn7h875w4w0000gn/T/ipykernel_27452/518383554.py:
17: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA
entries is deprecated. In a future version, this will no longer exclude empty
or all-NA columns when determining the result dtypes. To retain the old behavior,
exclude the relevant entries before the concat operation.
    corr_table = pd.concat([corr_table, corr_row]) # add each store's block to the
dataframe

```

	control_store	correlation	mag_measure	scores
0	40.0	1.0	0.936353	0.968176
1	26.0	1.0	0.914040	0.957020
2	58.0	1.0	0.906194	0.953097
3	72.0	1.0	0.895831	0.947915
4	81.0	1.0	0.889329	0.944665

```

/var/folders/ly/5zvv7mxn60s_ykmn7h875w4w0000gn/T/ipykernel_27452/518383554.py:
17: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA
entries is deprecated. In a future version, this will no longer exclude empty
or all-NA columns when determining the result dtypes. To retain the old behavior,
exclude the relevant entries before the concat operation.
    corr_table = pd.concat([corr_table, corr_row]) # add each store's block to the
dataframe
/var/folders/ly/5zvv7mxn60s_ykmn7h875w4w0000gn/T/ipykernel_27452/518383554.py:
17: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA
entries is deprecated. In a future version, this will no longer exclude empty
or all-NA columns when determining the result dtypes. To retain the old behavior,
exclude the relevant entries before the concat operation.
    corr_table = pd.concat([corr_table, corr_row]) # add each store's block to the
dataframe

```

From the above output, the stores with the highest scores are: 1 - Store 233 for trial store 77 2 - Store 155 for trial store 86 3 - Store 40 for trial store 88

Now, we want to see if there has been an uplift in overall chip sales. We'll start with scaling the control store's sales to a level similar to control for any differences between the two stores outside of the trial period.

```

In [34]: # Calculate the scaling factor for the store pairs
scale_store77 = pretrial_metrics[pretrial_metrics.STORE_NBR == 77]['totSales']
scale_store86 = pretrial_metrics[pretrial_metrics.STORE_NBR == 86]['totSales']
scale_store88 = pretrial_metrics[pretrial_metrics.STORE_NBR == 88]['totSales']

```

Scale pre-trial control sales to match pre-trial trial store sales

```

In [35]: # Extract the control store data from the df and scale according to the store
scaled_control233 = metrics[metrics.STORE_NBR.isin([233])][['STORE_NBR', 'YEARMO']]
scaled_control233.totSales *= scale_store77
scaled_control155 = metrics[metrics.STORE_NBR.isin([155])][['STORE_NBR', 'YEARMO']]
scaled_control155.totSales *= scale_store86
scaled_control40 = metrics[metrics.STORE_NBR.isin([40])][['STORE_NBR', 'YEARMO']]
scaled_control40.totSales *= scale_store88

```

```

# Combine the scaled control stores to a single df
scaledsales_control = pd.concat([scaled_control233, scaled_control155, scaled_
scaledsales_control = scaledsales_control.rename(columns = {'totSales': 'scaled_
# Get the trial period of scaled control stores
scaledsales_control_trial = scaledsales_control[(scaledsales_control.YEARMONTH:

# Get the trial period of the trial stores
trialsales = metrics[metrics.STORE_NBR.isin([77,86,88])][['STORE_NBR', "YEARMON
trialsales = trialsales.rename(columns = {'STORE_NBR': 'TRIAL_NBR'})
trialsales_trial = trialsales[(trialsales.YEARMONTH >= 201902) & (trialsales.YI

```

Now that we have comparable sales figures for the control store, we can calculate the percentage difference between the scaled control sales and the trial store's sales during the trial period.

```

In [36]: # Calculate the percentage difference between the control and trial store pairs
percentdiff = scaledsales_control.copy()
percentdiff[['TRIAL_NBR', 'tot_sales_t']] = trialsales[['TRIAL_NBR', 'totSales
percentdiff = percentdiff.rename(columns = {'scaled_tot_sales' : 'scaled_sales_
percentdiff['sales_percent_diff'] = (percentdiff.tot_sales_t-percentdiff.scaled
                                     /(0.5*((percentdiff.scaled_sales_c+percento

```

```

In [37]: # Display the percent difference
percentdiff

```

Out[37]:

	CONTROL_NBR	YEARMONTH	scaled_sales_c	TRIAL_NBR	tot_sales_t	sales_percent_diff
0	233.0	201807	281.980841	77.0	268.40	-0.049351
1	233.0	201808	271.063441	77.0	247.50	-0.090880
2	233.0	201809	229.681297	77.0	216.80	-0.057701
3	233.0	201810	165.632551	77.0	194.30	0.159293
4	233.0	201811	214.708863	77.0	224.90	0.046365
5	233.0	201812	275.950277	77.0	255.20	-0.078133
6	233.0	201901	156.482731	77.0	188.40	0.185091
7	233.0	201902	229.473346	77.0	211.60	-0.081045
8	233.0	201903	187.779277	77.0	255.10	0.304014
9	233.0	201904	149.932291	77.0	258.10	0.530192
10	233.0	201905	324.506712	77.0	272.30	-0.174954
11	233.0	201906	204.831215	77.0	246.60	0.185050
12	155.0	201807	875.427669	86.0	851.00	-0.028299
13	155.0	201808	718.052875	86.0	726.85	0.012177
14	155.0	201809	913.337595	86.0	855.00	-0.065980
15	155.0	201810	888.453131	86.0	898.80	0.011579
16	155.0	201811	811.661230	86.0	851.20	0.047555
17	155.0	201812	777.445092	86.0	812.20	0.043727
18	155.0	201901	811.272410	86.0	800.60	-0.013242
19	155.0	201902	827.019610	86.0	872.80	0.053865
20	155.0	201903	745.561872	86.0	945.40	0.236360
21	155.0	201904	778.028321	86.0	804.00	0.032833
22	155.0	201905	839.121625	86.0	826.90	-0.014672
23	155.0	201906	739.535166	86.0	766.00	0.035157
24	40.0	201807	1321.021376	88.0	1218.20	-0.080987
25	40.0	201808	1209.754860	88.0	1242.20	0.026465
26	40.0	201809	1354.315411	88.0	1361.80	0.005511
27	40.0	201810	1290.090144	88.0	1270.80	-0.015065
28	40.0	201811	1071.208716	88.0	1311.40	0.201620
29	40.0	201812	1302.763357	88.0	1213.00	-0.071361
30	40.0	201901	1283.646137	88.0	1215.40	-0.054618
31	40.0	201902	1346.153002	88.0	1339.60	-0.004880
32	40.0	201903	1273.550526	88.0	1467.00	0.141176
33	40.0	201904	1220.924472	88.0	1317.00	0.075712
34	40.0	201905	1365.592422	88.0	1236.85	-0.098940

	CONTROL_NBR	YEARMONTH	scaled_sales_c	TRIAL_NBR	tot_sales_t	sales_percent_diff
	35	40.0	201906	1455.915916	88.0	1252.60
						-0.150131

Calculate the t-values for the trial months. After that, find the 95th percentile of the t distribution with the appropriate degrees of freedom

```
In [39]: storepair = [[77, 233], [86, 155], [88, 40]]
pretrial_percentdiff = percentdiff[percentdiff.YEARMONTH < 201902]
pretrial_percentdiff_std = pretrial_percentdiff.groupby(['TRIAL_NBR'])['sales_percent_diff'].std()
dof = 6 # 7 months of data - 1

for stores in storepair: # stores numbers are stored as [trial, control] in storepair
    trialstore = stores[0]
    controlstore = stores[1]
    pretrial = percentdiff[(percentdiff.YEARMONTH < 201902) & (percentdiff.TRIAL_NBR == trialstore)]
    std = pretrial['sales_percent_diff'].agg('std')
    mean = pretrial['sales_percent_diff'].agg('mean')
    trialperiod = percentdiff[(percentdiff.YEARMONTH >= 201902) & (percentdiff.TRIAL_NBR == trialstore)]
    print("Trial store -", trialstore, "; control store -", controlstore)
    print("Month : t-statistic")
    for month in trialperiod.YEARMONTH.unique():
        xval = trialperiod[trialperiod.YEARMONTH == month]['sales_percent_diff']
        tstat = ((xval - mean)/std)
        print(str(month), ' : ', tstat)
    print()
```

```
Trial store - 77 ; control store - 233
Month : t-statistic
201902 : -0.8435806220494315
201903 : 2.4904464734755805
201904 : 4.448806995302806
```

```
Trial store - 86 ; control store - 155
Month : t-statistic
201902 : 1.3077596199049661
201903 : 5.828582596615282
201904 : 0.7867590051021528
```

```
Trial store - 88 ; control store - 40
Month : t-statistic
201902 : -0.06747939544188036
201903 : 1.4413280336576249
201904 : 0.7650631287827934
```

Conclusion

In this task, we found that the results for trial stores 77 and 86 showed a statistically significant difference in at least two stores of the three months of the trial period. However, this was not the case for store 88. We can check to see if the trial was implemented differently in store 88 but even so, we have been able to see that the trial has resulted in a significant increase in sales.

In []: